Introduction to MongoDB

01

Techna! I wonder how the supermarket tracks the records of thousands of products.

It is through database management system. Large offices or supermarkets use databases to store and manage data effectively. It is less time consuming and provides access to multiple users. Databases also provide security to its users. Let us understand database through this lesson.

Introduction

In this lesson, we are going to get familiar with databases, specifically MongoDB. We will understand what databases are, how to store data, and other related functionalities.

Before we get started with Database, let us define Data.

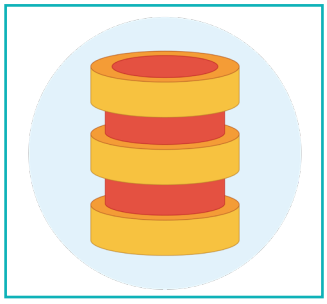
What is Data?

Data is just small bits of information related to an object available in different formats, such as integers, media, texts, and so on. It can be anything such as price, cost, names of products, date, time and place of events, your name, address, age, etc.

Storing data on a personal computer is an easy task. However, in large organisations, the amount of data is huge such as employee names, salaries, employee details, etc. Storing and managing such huge data is a tedious task. Here, the database comes into action. Let us understand what a database is.

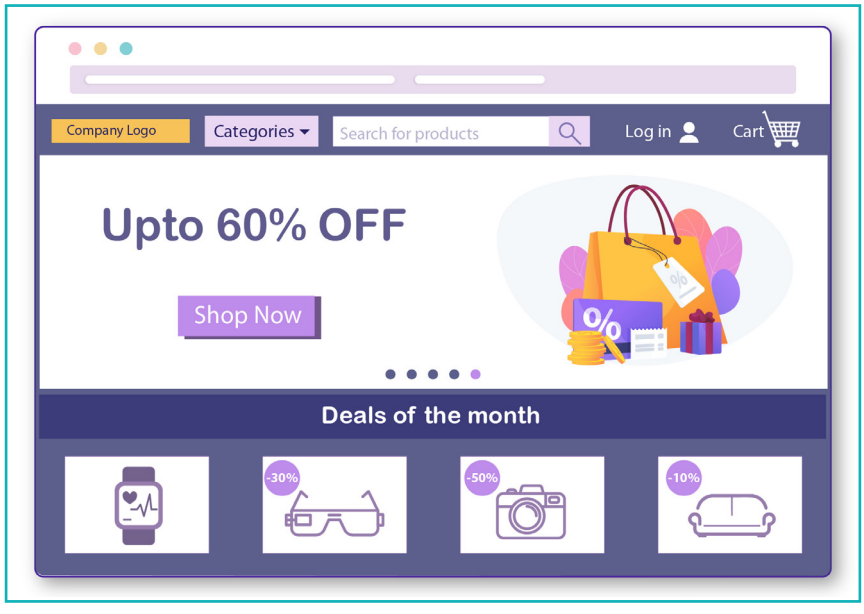
Database

A **Database** is a structured collection of related data/information. All the data in a database are related to each other hence, it is called 'organised data'. For example, a school's database should include all student-related information such as student marks, subjects, subject teachers, and grades.



While browsing on the internet, we interact with databases unknowingly. Many websites ask us for details such as name, email-id, address, etc. for registration purpose. After registration, all this data is stored in a database so that the user can access it anytime.

Consider an example of an e-commerce application that lists numerous products. The e-commerce applications store product name, product images, prices, size details, and others. All these data of the numerous products are stored in the database. Databases store data electronically in a computer system so that it may be conveniently accessed and controlled.



Databases serve several advantages, such as:

- Easy to store and retrieve records within seconds.
- Thousands of users can access the same database from different locations at the same time.
- Database also provides security to our data, as login credentials are required to access the data stored in the database.

Applications of Database

Database is used in various fields to store and manage data effectively. Some of the most common applications of databases are as follows:

- **Telecom:** Telecom industries store information regarding call details, network, customer details, etc.
- **Banking:** Banks store customer information, daily transactions, loan information, account information, etc.
- **Railway:** Railway uses databases to store information such as ticket allotment, passenger information, train schedules, timings, etc.
- **Social Media:** Social media industry uses databases to store information related to user accounts, uploaded posts and images, daily activities, etc.

Apart from these fields, database is also used in various other sectors such as Education, Aviation, Employee Management, Online Shopping, Manufacturing, and others.

Types of Databases

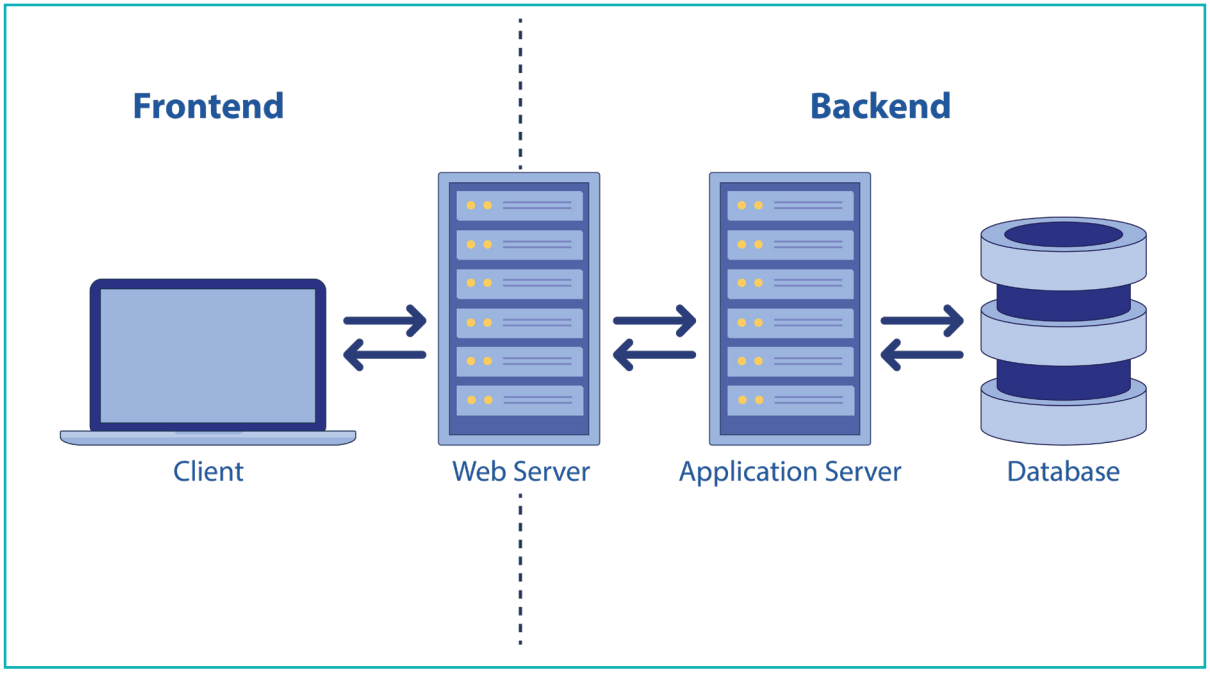
Data is stored in different types of databases depending on the type, structure, and quantity of data. The most common types of databases are:

1. Relational Database
2. Object Oriented Database
3. Distributed Database
4. NoSQL Database
5. Graph Database

Let us now understand how to work with the databases.

Database Management System (DBMS)

Database Management System is a type of software used to create, access, update, and extract data from a database in an organised way. It provides an end-user interface to easily manage the database by storing the data systematically. It also facilitates various other functionalities, such as monitoring performance, searching data, creating backups and recovery. To summarise, a database is controlled by a Database Management System. Some popularly used DBMS are– MySQL, Microsoft SQL Server, dBase, MongoDB, etc.



Exercise

Tick mark ☑ the correct answer

1. Which of the following is not a type of Database?

   a. Relational Database ☐ **b.** Object Oriented Database ☐

   c. NoSQL Database ☐ **d.** Programming Database ☐

2. What is the full form of DBMS?

   a. Database Modelling System ☐ **b.** Database Management System ☐

   c. Data Manager System ☐ **d.** Data Making System ☐

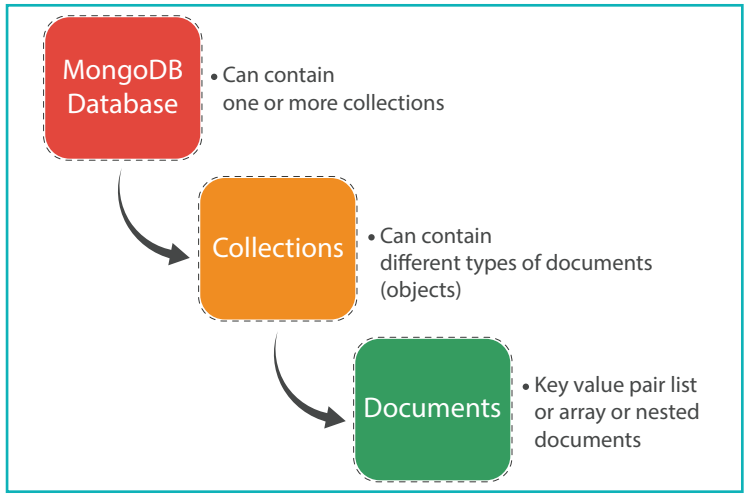
In this lesson, we will create a database using MongoDB.

MongoDB

MongoDB is a Database Management System for storing and managing data. It is categorised as a NoSQL database. It is a document-based database that stores data as Collections and Documents. It is designed to store huge volumes of data and provide efficient data management. MongoDB can be used in several programming languages, such as C, C++, Python, Java, JavaScript, etc.

In the MongoDB database, the data hierarchy starts with the database. A database can have multiple Collections, each of which contains one or more documents. Each document again stores data in the form of fields.

The following figure depicts the data hierarchy in a MongoDB Database.

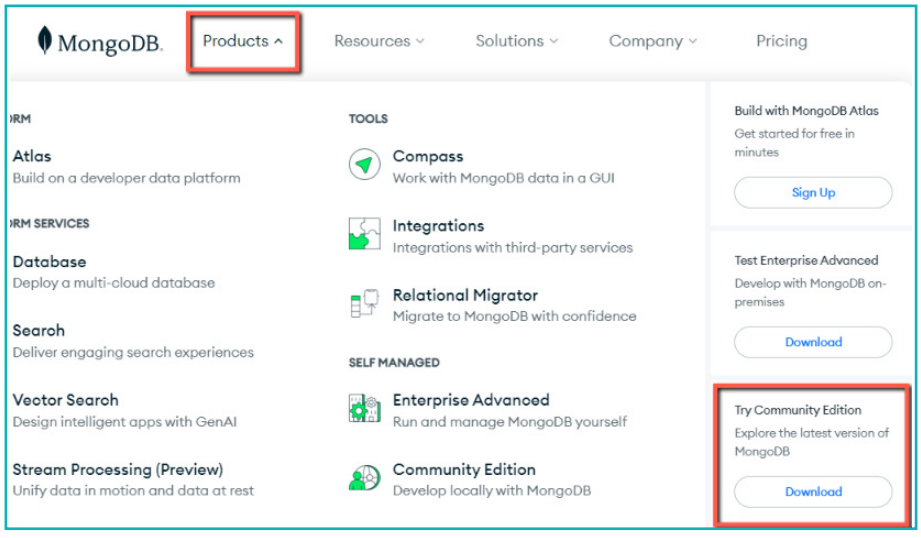


Let us start with the project to understand these terminologies and purpose.

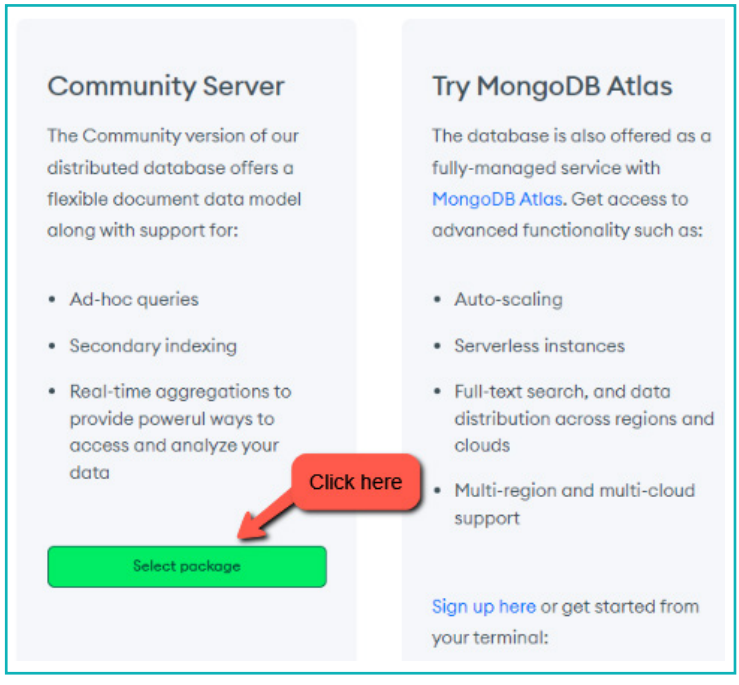
Download and Install MongoDB

1. To get started with MongoDB, the first thing we need to do is download it.
Visit https://www.mongodb.com/ on your browser. Alternately, you can also search for MongoDB on Google.

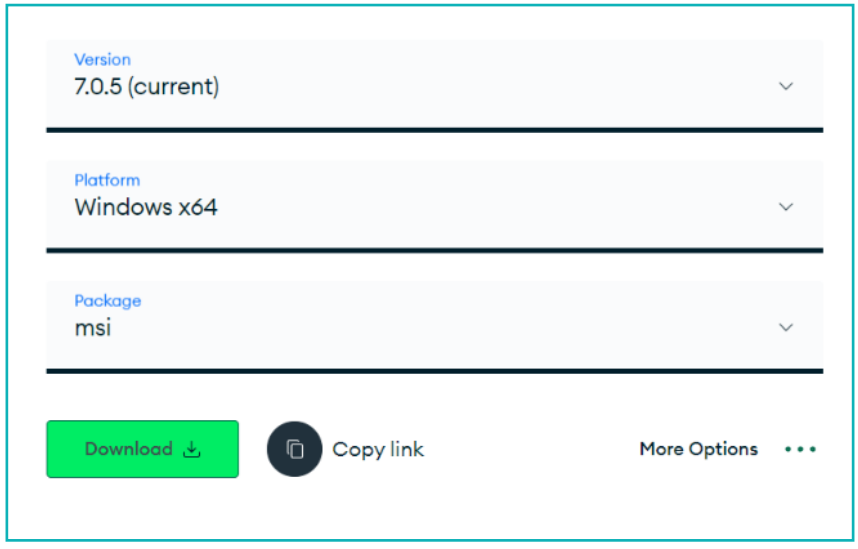
- Once you are on the website, click on the Products tab and under 'Try Community Edition', click Download.



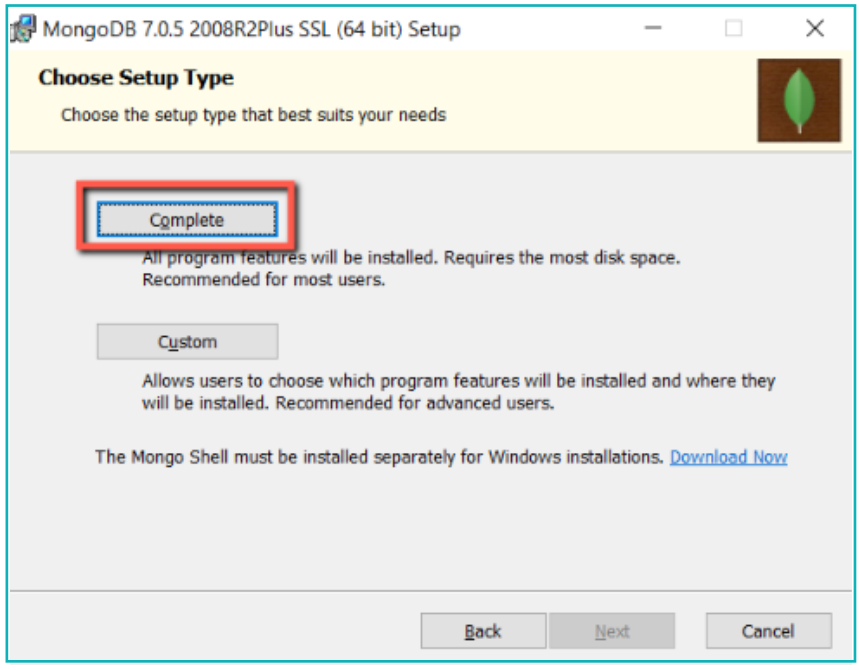
2. On the next page, click on the 'Select Package' button.



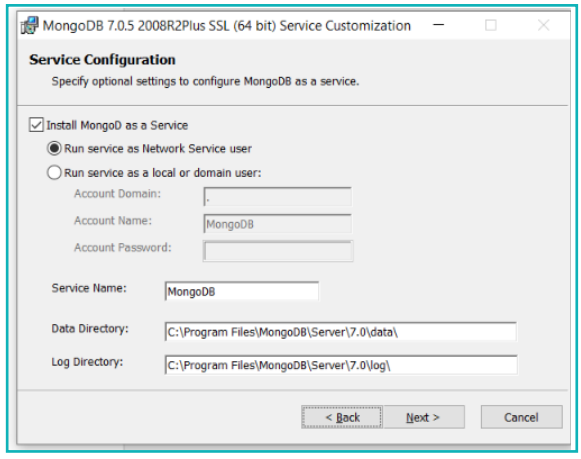
3 Now, select the appropriate platform and click on the “Download” button



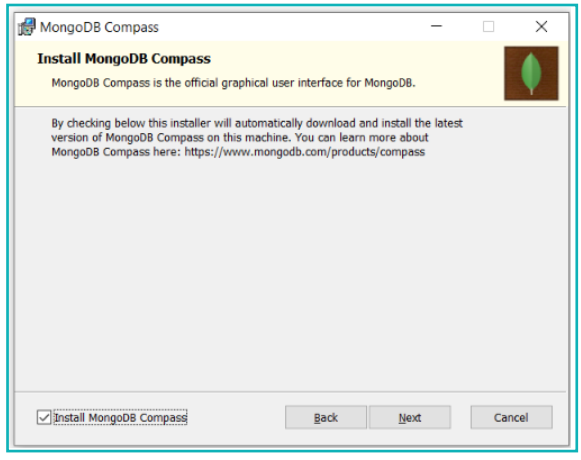
4 Once the download completes, install the application by double clicking on the downloaded setup file. Accept the provided “Terms in the License Agreement” and choose the Complete option. This will install all the required components on our system.



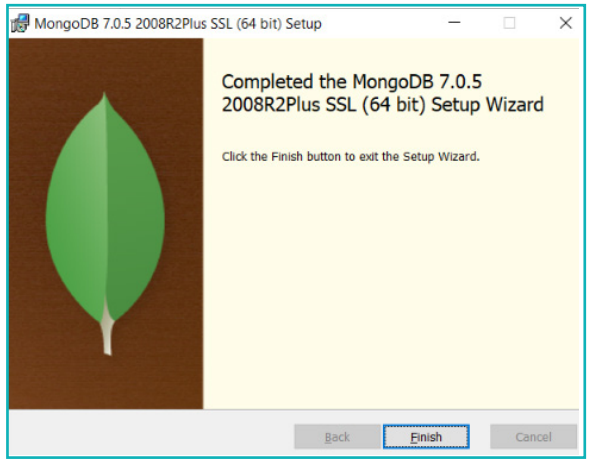
5 For the next option, we will go ahead with the default settings.



5 Click 'Next' and then, click on the 'Install' button in the next window.



The installation process might take a few minutes. Once done, click on the Finish button.



Note:
Depending on the time when you are installing the application, your MongoDB version might be different than the one shown here.

Before we proceed, it is important to understand that there are 2 ways to work with MongoDB–

The first option is 'MongoDB Shell', which is 'Command Line Interface (CLI)', where we type commands to perform various database-specific operations.

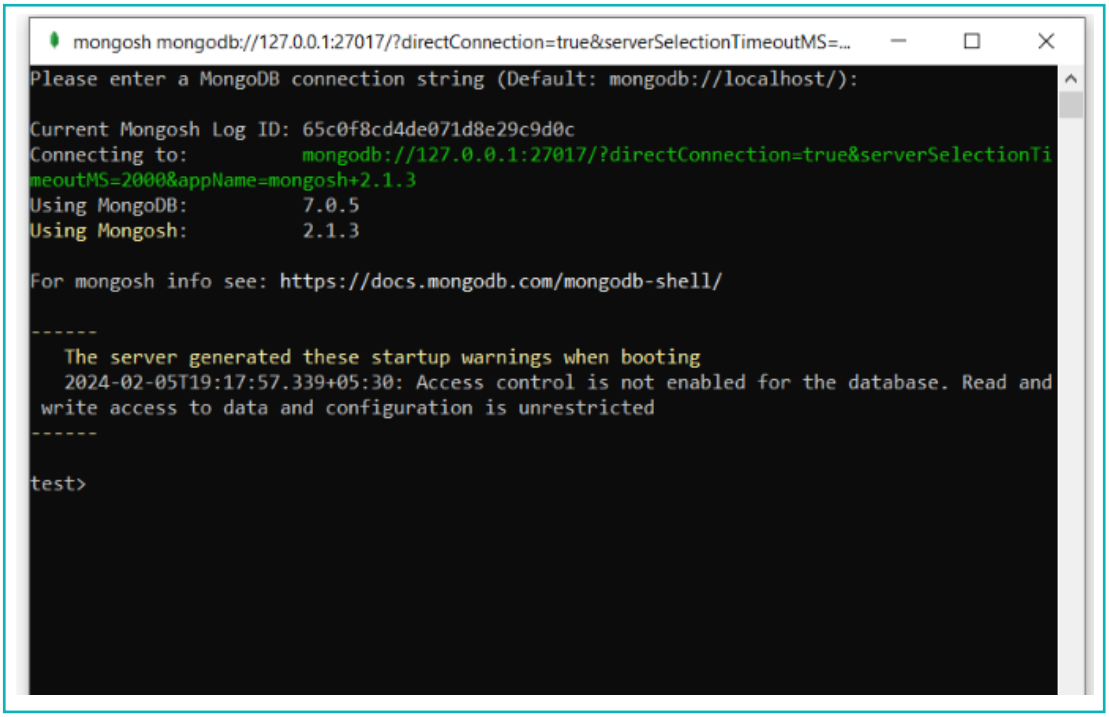


Figure: MongoDB Shell

The other option is MongoDB Compass, which provides a graphical interface for us to work with the Database. Every operation that we can do using MongoDB Shell can be done using MongoDB Compass, as well. The difference among the two is that in MongoDB Shell, we need to type various commands, whereas, in MongoDB Compass, we are presented with a Graphical Interface to work with the database.

For this lesson, we are going to use MongoDB Shell to operate and manage the database.

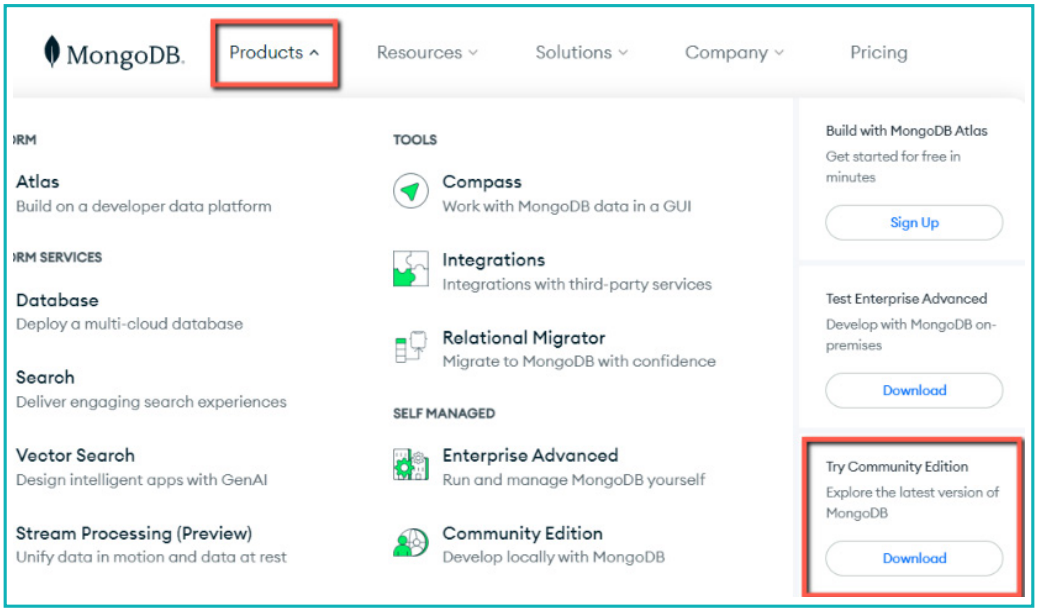
Download and Install MongoDB Shell

At this point, we have MongoDB installed on our system. However, MongoDB Shell doesn't come along with it. To use MongoDB Shell, we need to download and install it separately.

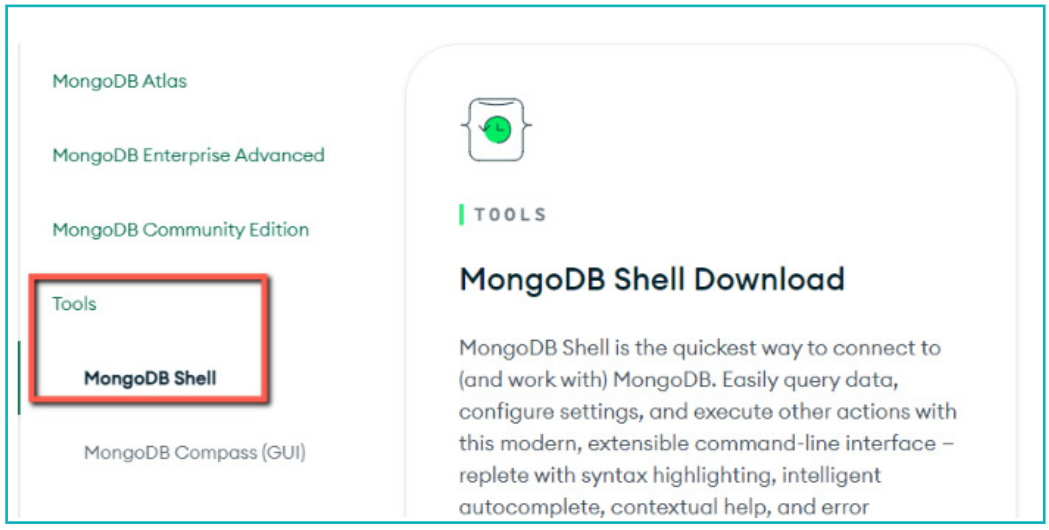
What is MongoDB Shell?

MongoDB Shell is a Command Line Interface **(CLI)** that allows us to interact with the MongoDB database through the command line. This shell can be used for various purposes such as creating and managing databases, manipulating data, etc. It is the default interface for working with MongoDB and since it is CLI, every database operation is executed by typing commands on the shell.

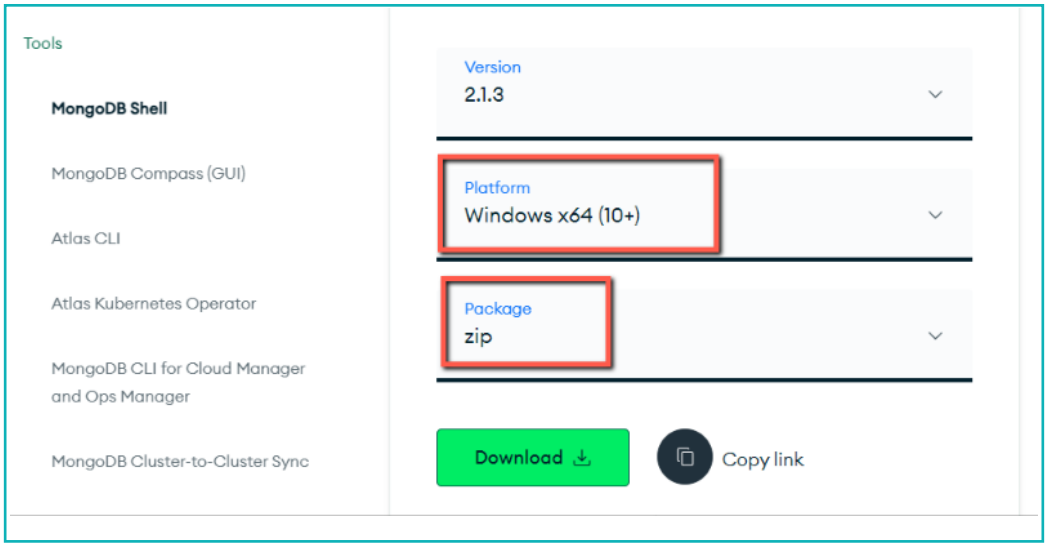
1. To install 'MongoDB Shell', visit the MongoDB website and click on the 'Products' tab and under 'Try Community Edition', click Download. This will take you to the 'MongoDB Community Server' section.



2. In the newly opened window, click on 'Tools' present on the Sidebar and then click on 'MongoDB Shell'.

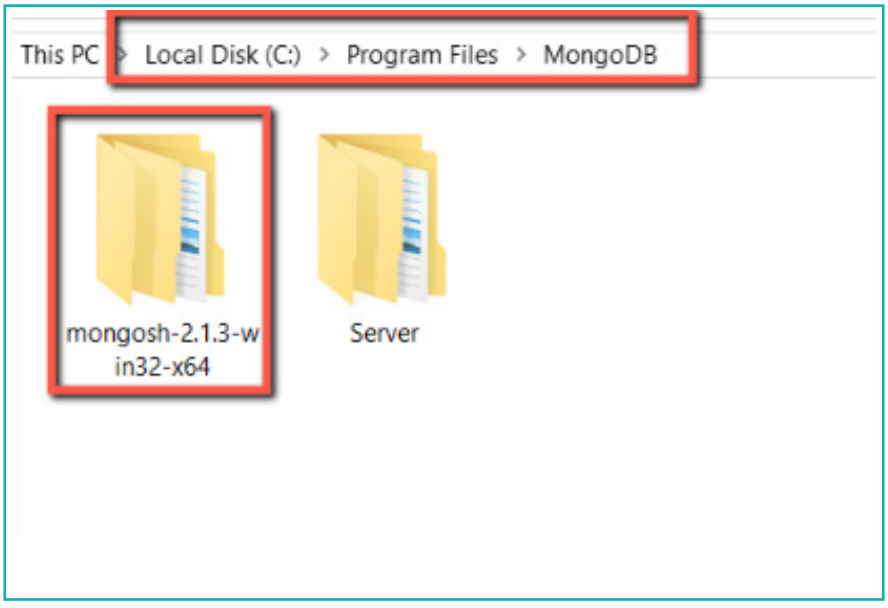


3 Scroll down on the 'MongoDB Shell Download' page and you will find the option for Download. Next, select your Platform and click on the Download button. The Version and Package can be kept the way they are.

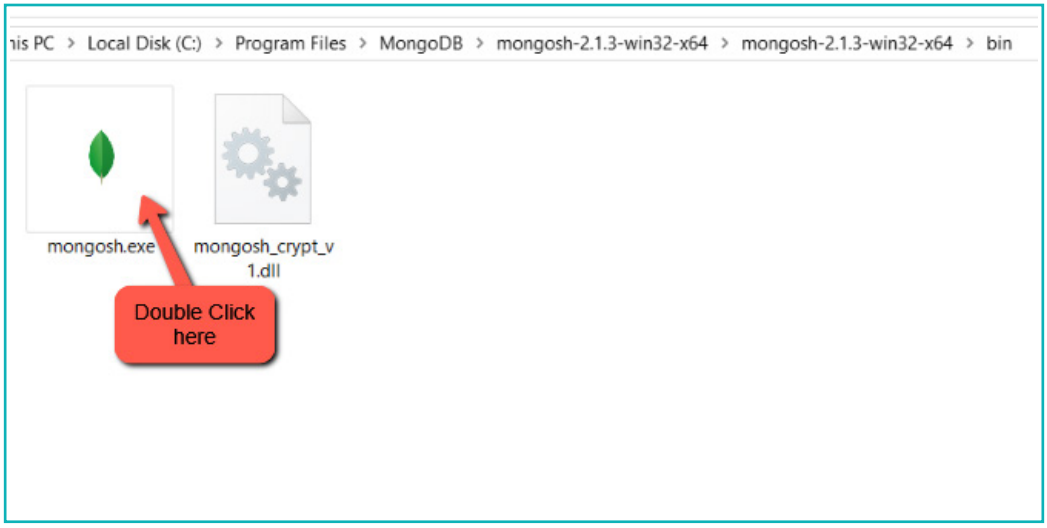


The downloaded Mongo Shell file will be available in a .zip format, which we need to extract by right-clicking on the .zip file and selecting the Extract All option. Once extracted, a new folder will be created with the same name as the .zip file.

4 By default, MongoDB gets installed in the “C:\Program Files\MongoDB” directory and so we will move the extracted folder to this location.

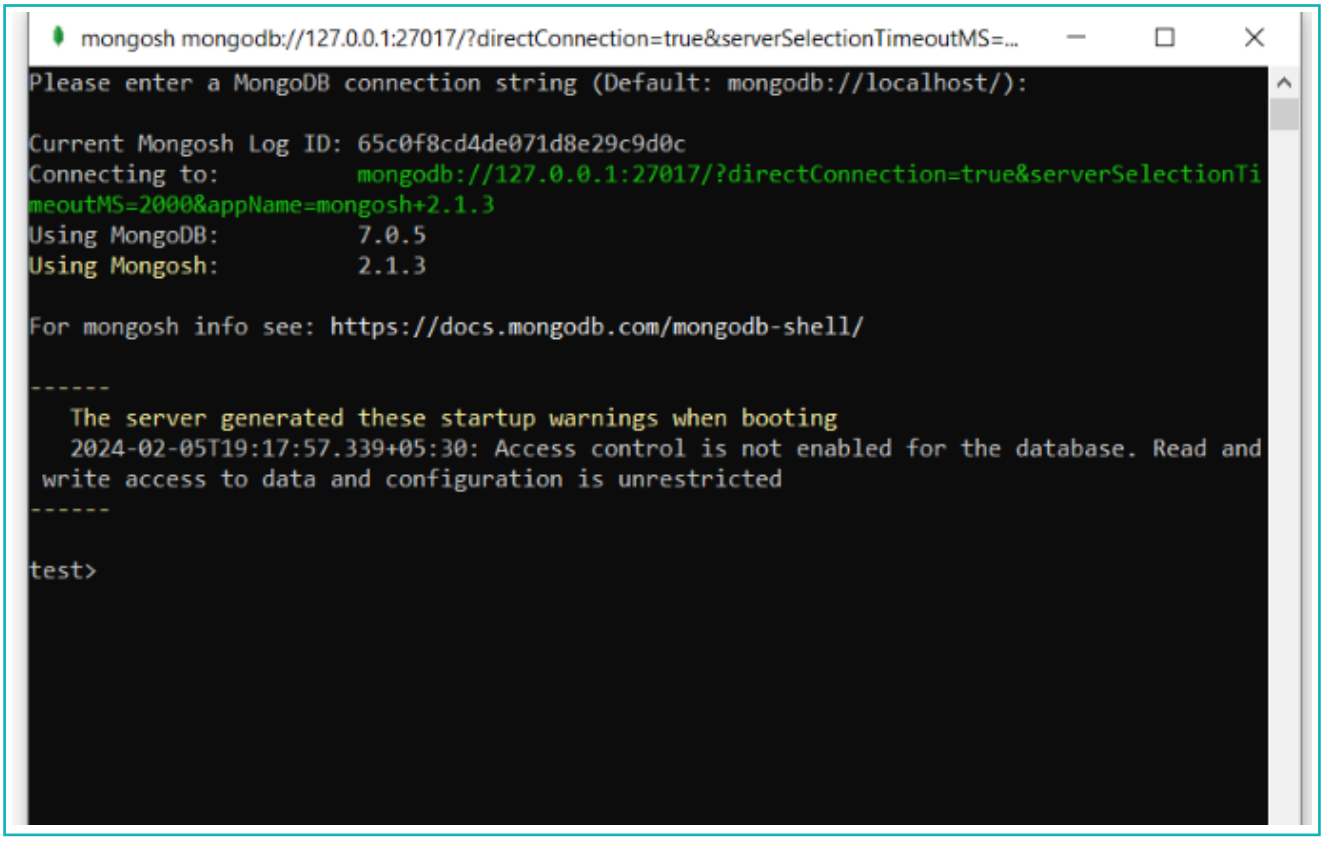


5 Within this folder, open the ‘bin’ folder and double-click on the ‘mongosh.exe’ file to run the MongoDB Shell.



6 When the Shell opens, it will display the message “Please Enter a MongoDB connection string”. Since we will be using the default connection string, simply press ‘Enter’.

The Mongo Shell CLI is now ready to use.



Exercise

State if the statements are True or False

3. CLI stands for Control Line Interface.

▲ _____

4. "MongoDB Shell" provides a graphical interface to work with a database.

▲ _____

Create a Database

To store any form of data in MongoDB, we need to create a database. However, before creating one, let us understand the purpose of creating it.

Suppose that we want to create a database for storing the details of students and teachers at school. We will name the database "SchoolDB" and for storing the details of students, we will create a Collection named "Student".

> **Note:**
> You can also create a separate Collection called "Teachers" for storing the details of Teachers in the school.

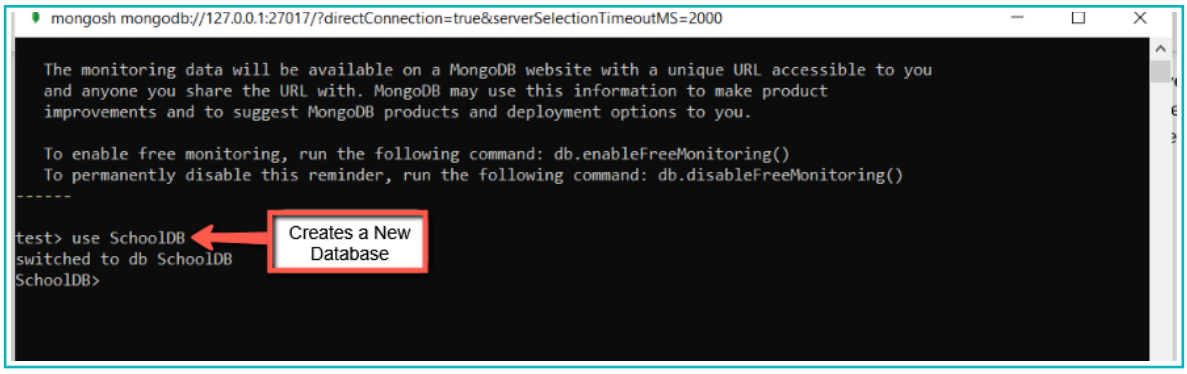
Now that we have decided which data we need to store, let us create the Database.

MongoDB does not have a 'create database' command; rather, it is created using the 'use' command.

Use Command

The 'use' command is mainly used to connect with an existing database. For example, if we want to connect with a database named 'AdminDB', we need to use the command, "use AdminDB". However, if we execute the 'use' command to connect with a database and that database does not exist, then it creates a new database with the same name and connects with it as well.

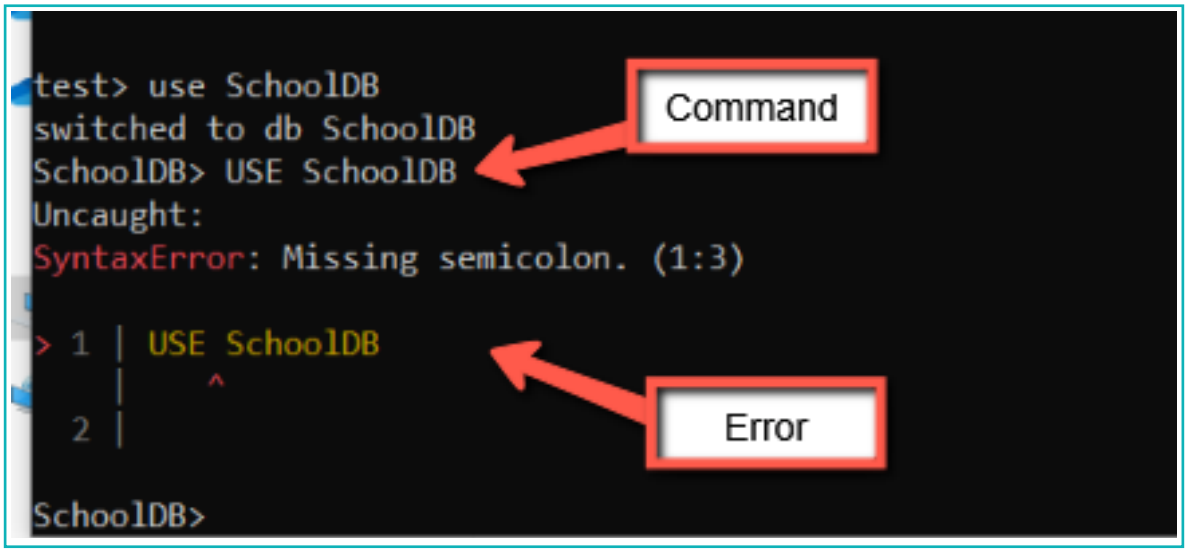
1. Execute the 'use' command to create and connect with a new database "SchoolDB".



Note:
The commands we execute on the Shell are case-sensitive, which means that the command written in lowercase and uppercase are processed differently.

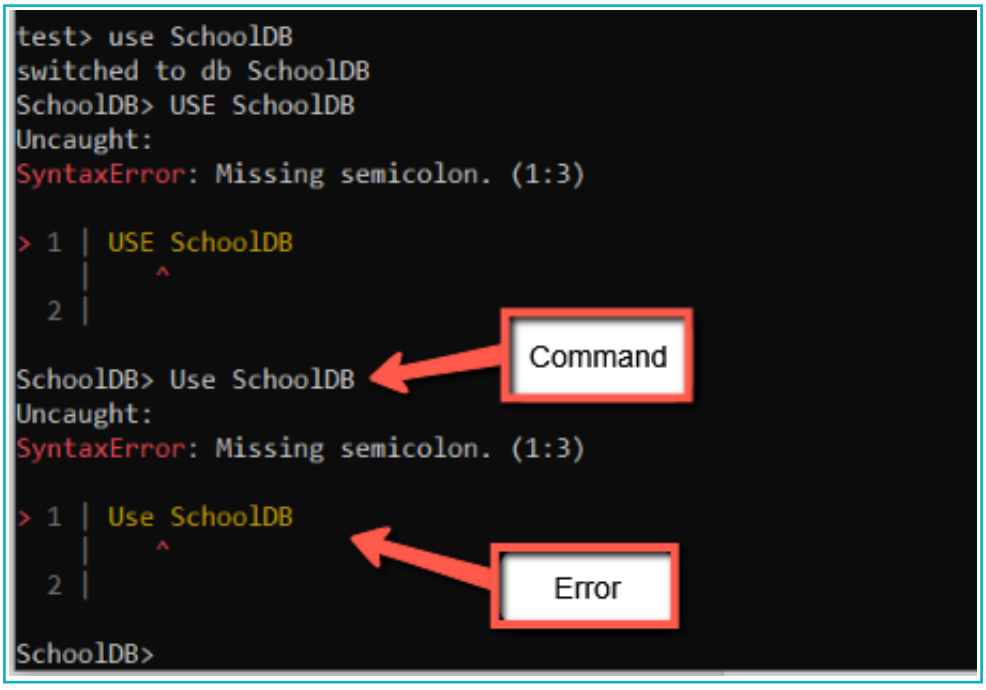
Let us understand with the help of an example-

2. In the Shell, execute the command, "USE SchoolDB". Notice the command 'use' is written in uppercase.



Executing the 'USE SchoolDB' command in uppercase results in an error.

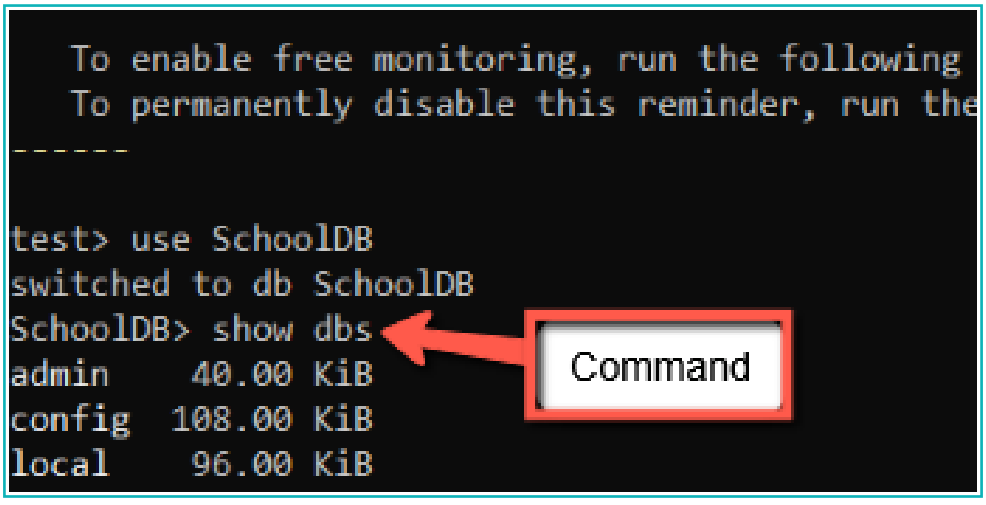
3 Next, let us also try changing the case of the first letter in the 'use' command and checking if any error pops up or not.



Even now, we get an error. This implies the importance of using the proper case while writing down 'Shell Commands'. So, from this, we can state that MongoDB Shell Commands are case-sensitive.

Now, we will display all the databases that are available on MongoDB Server.

- To check if the database is created or not, we can execute the 'show databases' or 'show dbs' command. This command displays a list of all databases present in the MongoDB Server.

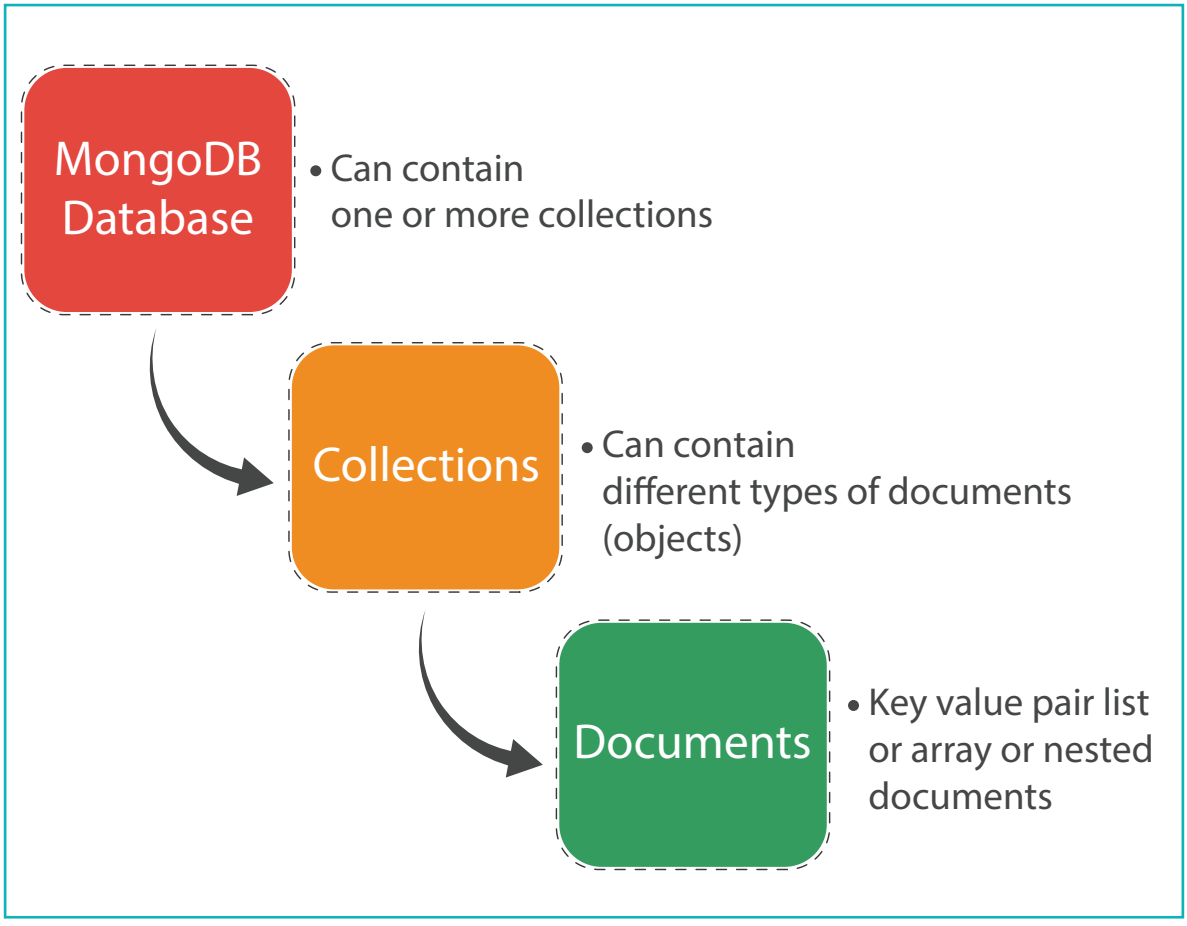


The list of databases does not contain the 'SchoolDB' database (the one we created). This is because MongoDB does not display empty databases. Since our 'SchoolDB' database does not contain any data yet, it is not shown on the 'Database' list. The other databases, 'admin', 'config' and 'local' are default databases that comes along with MongoDB. We can ignore those databases, as they are not required by us.

To understand how to store data on our database, we need to get familiar with the concept of 'Collections' and 'Documents'.

Collections and Documents in MongoDB

We cannot store data directly in the database. Instead, data is stored in Collections that are further stored in the database. A single database can contain multiple collections and each collection can have multiple documents.

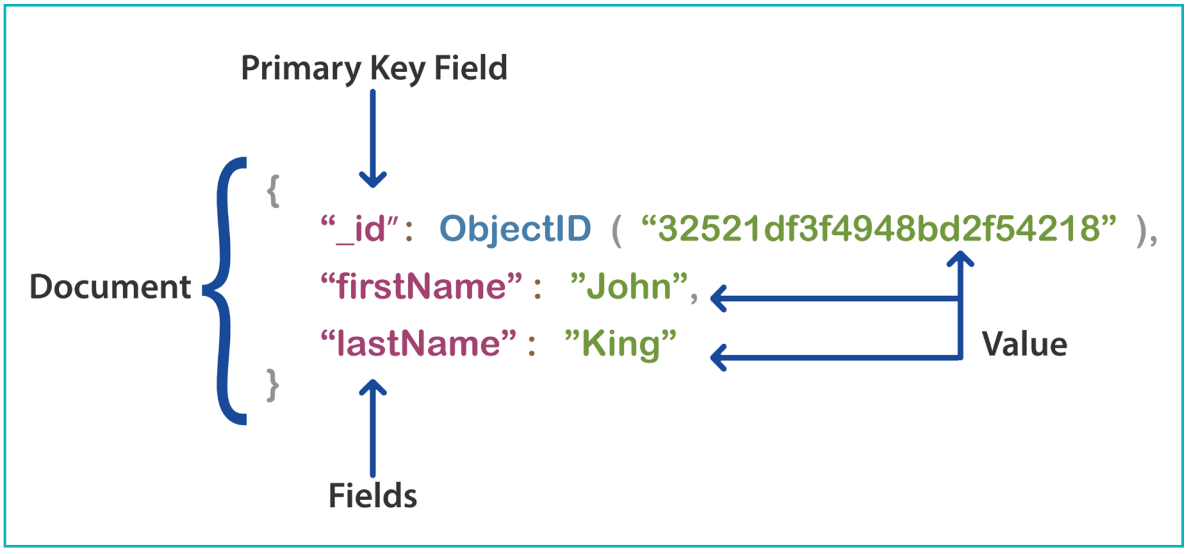


Documents in MongoDB

Documents refer to the records of data that we store on a database. Data in a document are stored in 'field-value' pairs and have the following structure:

```
{
    field1: value1,
    field2: value2,
    ...
    fieldN: valueN
    }
```

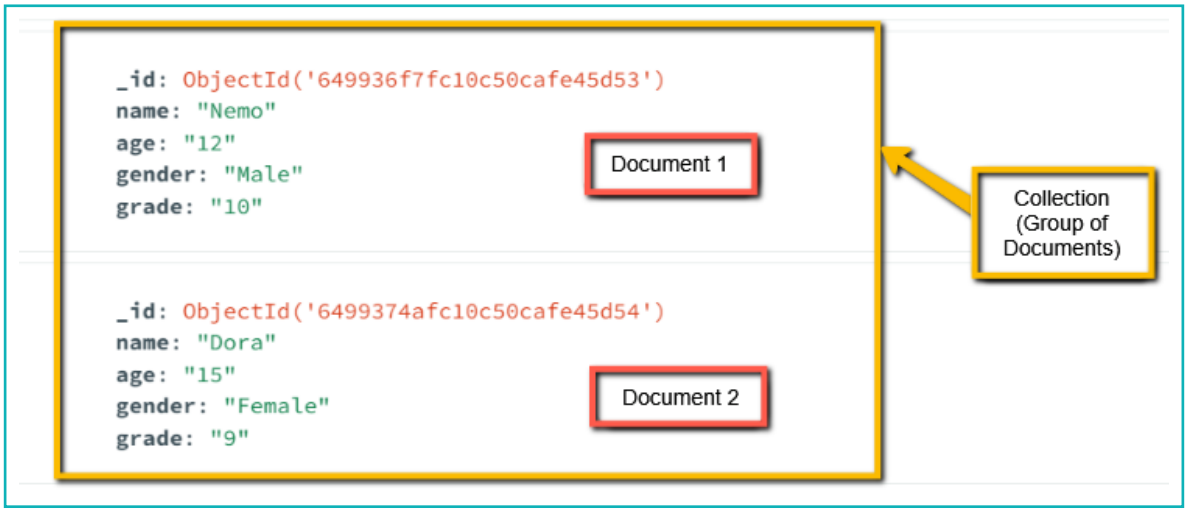
The following example shows a sample document that stores the 'First Name' and 'Last Name' of a person.



Notice the '_id' field and its associated value. This field is known as the 'Primary Key' field and every document that we create has this field. This field is automatically inserted whenever a document is created and contains a unique value for each document.

Collections in MongoDB

In MongoDB, a 'Collection' is a group of MongoDB Documents. It is part of a Database, and a single Collection can hold multiple Documents. For example, we can have a Collection named 'Students', which can store the Records (Documents) of all the students.



To summarize,

- **Database** contains one or more **Collections.**
- **Collections** contain one or more **Documents.**
- **Documents** contain the data/information that we want to store.

Create Collections

Our database is already created, so now we will create the Collection 'Student' on the database.

To create a Collection, the command 'db.createCollection()' is used. Its syntax is as follows-

db.createCollection("<Collection_name>")

1. Let us create a Collection ‘Student’.

```
test> use SchoolDB
switched to db SchoolDB
SchoolDB> show dbs
admin    40.00 KiB
config  108.00 KiB
local    96.00 KiB
SchoolDB> show databases
admin    40.00 KiB
config  108.00 KiB
local    96.00 KiB
SchoolDB> db.createCollection("Student")
{ ok: 1 }
SchoolDB>
```

Note that after executing the command we are getting an “ok: 1” response, which states that the Collection ‘Student’ is successfully created. Using this command, we can create as many Collections as necessary. However, for this project, we will create only a single Collection.

To view all the Collections, present on the database, we can use the command ‘show collections’.

```
SchoolDB> Use SchoolDB
Uncaught:
SyntaxError: Missing semicolon. (1:3)

> 1 | Use SchoolDB
    |     ^
  2 |

SchoolDB> show collections
Student
SchoolDB>
```

In the output, we can see that our database contains a single collection ‘Student’.

Create Documents

The collection we just created is currently empty. So let us store some data on this Collection, which means we will be creating 'Documents'.

In Documents, data is stored in JSON format.

Exercise

Tick mark ☑ the correct answer

5 Which command can be used to create a database using MongoDB Shell?

a. 'use' command ☐ **b.** 'create' command ☐

c. 'start' command ☐ **d.** 'init' command ☐

6 Which of the following statements is True?

a. In MongoDB, data is stored in Collections and Documents ☐

b. In MongoDB, data is stored in Tables and Rows ☐

c. A single MongoDB database can contain only 1 Collection ☐

d. Collections reside inside Documents ☐

What is JSON?

JSON stands for 'JavaScript Object Notation' and is a file format used to store the data to and from web applications. In JSON, data is stored in 'key-value' pairs, which makes it easy to use and understand.

The following is an example of data stored in JSON format. Here, 'title' and 'author' are keys and 'Alice in Wonderland' and 'Lewis Carroll' are values. Each 'key-value' pair is separated by a comma. Also note that while storing data, each key and its corresponding value must be wrapped within double quotes.

```
2
3   {
4       "title": "Alice in Wonderland",
5       "author": "Lewis Carrol"
6   }
7
8
```

Inserting a Single Document

To create/insert a single Document in a Collection, the command used is -

db.<Collection Name>.insertOne({

Key: value,

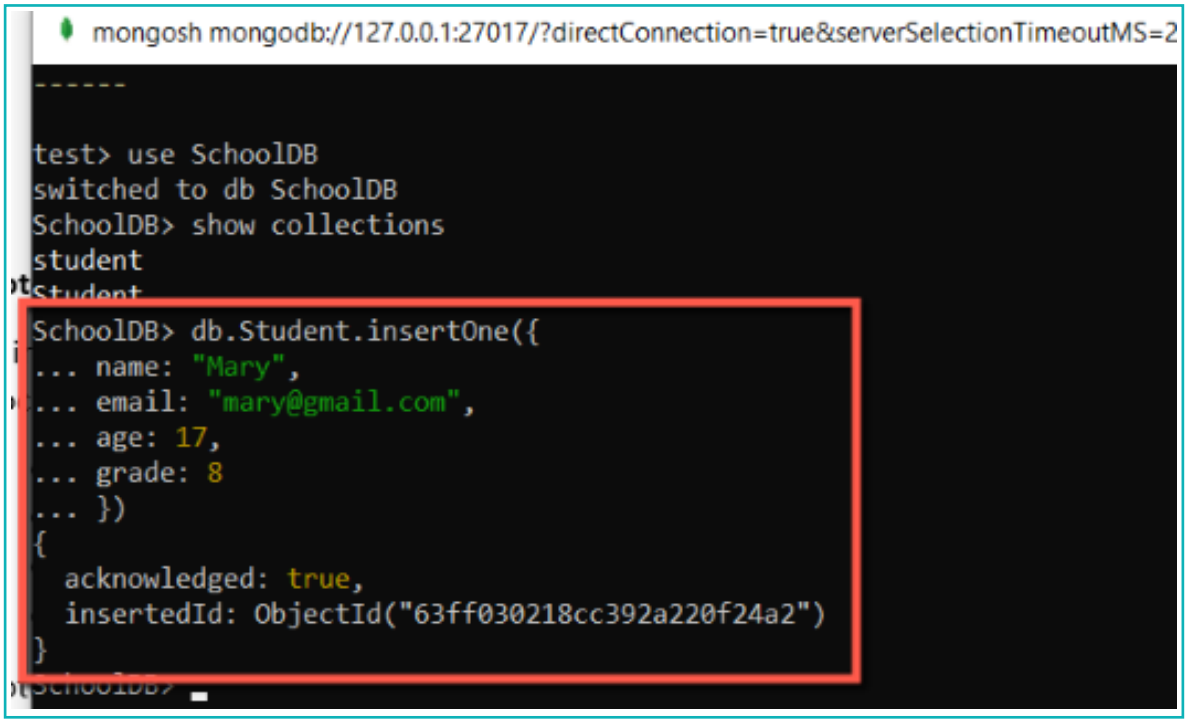
Key: value,

…..

…..

})

Let us use this command to insert a fresh document in our Student Collection.



Note:
Press 'Enter' after writing 'insertOne({' to move to the next line.

Using the 'insertOne()' command, we inserted a single document in our 'Student' Collection. The 'Document' inserted has 4 key-value pairs -

- name: Mary
- email: mary@gmail.com
- age: 17
- grade: 8

Notice that we are receiving an additional response from MongoDB. This response indicates that our insert operation was successful.

The response also contains 2 key-value pairs -

- "acknowledged: true" indicates that the method was executed successfully.
- "insertedId: ObjectId("63ff030218cc392a220f24a2")" represents the unique ID that is automatically assigned to the inserted document. Each document we insert will have a unique 'ObjectId' value. No two documents will have the same 'ObjectID' value.

A few points to consider while inserting values in a Document—

- Alphabetical values should be enclosed in double (“ “) or single(‘ ‘) quotes.
- Numerical values should not be enclosed in double (“ “) or single(‘ ‘) quotes.
- Each key-value pair should be separated by a comma, except the last pair.

In this manner, we can insert as many documents as needed. So, following the same process, let us insert another ‘Document’ into the ‘Student’ Collection.

```
{
  acknowledged: true,
  insertedId: ObjectId("63ff030218cc392a220f24a2")
}
SchoolDB> db.Student.insertOne({
... name: "Rahul",
... email: "rahul@gmail.com",
... age: 19,
... grade: 9
... })
{
  acknowledged: true,
  insertedId: ObjectId("63ff03c318cc392a220f24a3")
}
SchoolDB>
```

So now, we have 2 documents inserted into the ‘Student’ Collection.

Note:
Collection names are also case-sensitive, i.e., ‘Student’ (with capital S) and ‘student’ (with small s) refer to two different Collections.

Fetch the Documents from the Collection

To fetch all the documents stored in a Collection, MongoDB provides the 'find()' function.

The syntax is-

db.<Collection_name>.find()

1. Let us execute this command on the shell and check the output.

From the output, we can see that we were able to fetch the documents we had just inserted into the 'Student' Collection.

Search or Filter the Documents

The 'find()' method displays all the Documents in the Collection. However, in certain scenarios, we may need to fetch or search for a particular document.

For example, let's say we want to fetch the details of only "Rahul".

In such cases, we can use the 'find()' method and specify the key we want to search for as an argument to it.

Let us see how to use the 'find()' method.

1. Use the 'find()' method and specify the name: "Rahul" key-value pair as a parameter.

We can see that we got only 1 document, specific to the name filter we had used.

Update the Documents

In many cases, we might need to update an existing Document. To do so, MongoDB provides the 'updateOne()' method. This method requires a '$set' operator, where we need to specify the updated data.

Syntax-

db.<Collection_name>.updateOne({search_criteria}, {$set: {data_to_update} })

1. To demonstrate the functionality, we will update the 'age' of Mary from 17 to 18 using the 'updateOne()' method.

If the 'updateOne()' command is successful, we receive a response from MongoDB with the operation's status.

2. Now if we use the 'find()' method to view all the Documents, we should see the updated data.

Delete Documents

To delete a 'Document', MongoDB provides the 'deleteOne()' method.

Syntax -

db.<Collection_name>.deleteOne({search_parameter})

1. Let us delete the Document of 'Mary' using the 'deleteOne()' command.

```
SchoolDB> db.Student.deleteOne( {name: "Mary"} )
{ acknowledged: true, deletedCount: 1 }
```

2. Execute the 'find()' method to confirm that our 'delete()' method was successful.

```
SchoolDB> db.Student.find()
[ acknowledged: true, deletedCount: 1 }
  {
    _id: ObjectId("63ff03c318cc392a220f24a3"),
    name: 'Rahul',
    email: 'rahul@gmail.com',
    age: 19,
    grade: 9
  }
]
```

From the output, we can see that the document containing details of Mary is no longer available in the 'Student' Collection.

Exercise

Tick mark ☑ the correct answer

7 To create a Collection, the _____ command is used.

a. db.create() ☐ **b.** db.insertCollection() ☐

c. db.Collection() ☐ **d.** db.createCollection() ☐

8 Which format is used to store data in Documents in MongoDB?

a. XML ☐ **b.** JSON ☐

c. CSV ☐ **d.** None of the above ☐

In this lesson, we got familiar with the basics of MongoDB Shell and created databases, collections, and documents. We have implemented methods/commands such as use, update, delete, etc. We have understood the case-sensitivity property of the MongoDB Shell.

1. Introduction to MongoDB

Tech Flash

- **Document-based Database** : It is a type of NoSQL database in which data is stored in 'key-value' pairs rather than the traditional way of storing data in tables with pre-defined schemas.

Relational Database : A type of Database Management System (DBMS) that organizes and stores data in a tabular format, consisting of tables with rows and columns. Widely used in various industries due to their flexibility and ability to handle complex relations among data.

JSON : Short for JavaScript Object Notation, **JSON** is the most popular data storage format, commonly used for storing and sharing data among different applications. It gained popularity, as it is easy for humans to read and write JSON data as well as for machines to parse and generate output.